



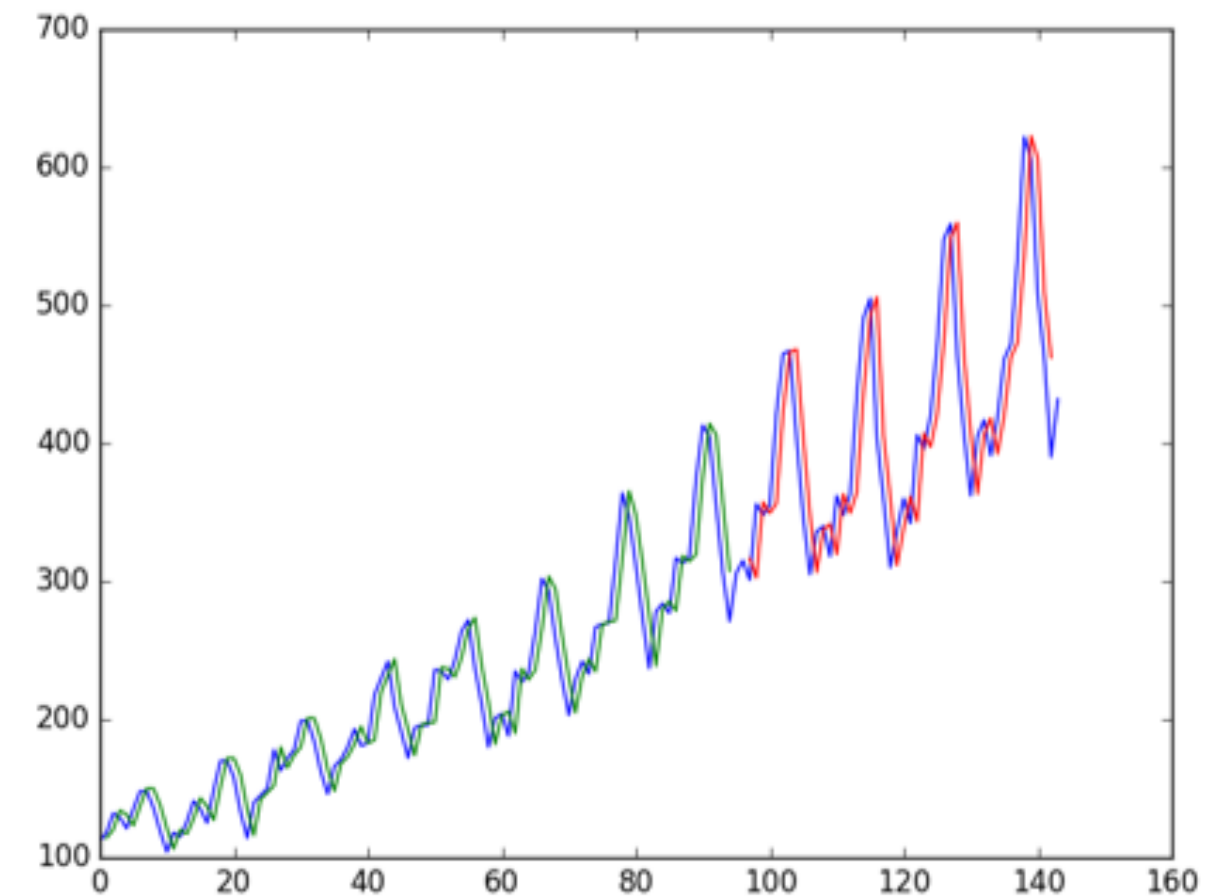
SERIES DE TIEMPO



DATOS DE SERIES DE TIEMPO

Registros de cambios diarios, mensuales, etc, organizados de más antiguos a más nuevos.

Están organizados, generalmente, en intervalos regulares.



```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import month_plot, quarter_plot
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.seasonal import seasonal_decompose
```

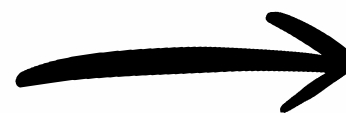
- **month_plot / quarter_plot:** comparar visualmente el comportamiento típico por mes o trimestre para detectar estacionalidad.
- **plot_acf:** muestra qué rezagos tienen correlación con la serie; útil para ver estacionalidad y dependencia temporal.
- **plot_pacf:** muestra la contribución “directa” de cada rezago; ayuda a elegir cuántos rezagos son relevantes.
- **seasonal_decompose:** separa la serie en componentes (tendencia, estacionalidad, residuo) para entender estructura y ruido.



	A	B	C	D	E	F	G
1	Date	Open	High	Low	Close	Adj Close	Volume
2	2014-09-17	465.8640137	468.1740112	452.4219971	457.3340149	457.3340149	21056800
3	2014-09-18	456.8599854	456.8599854	413.1040039	424.4400024	424.4400024	34483200
4	2014-09-19	424.1029968	427.8349915	384.5320129	394.79599	394.79599	37919700
5	2014-09-20	394.6730042	423.29599	389.8829956	408.9039917	408.9039917	36863600
6	2014-09-21	408.0849915	412.4259949	393.1809998	398.8210144	398.8210144	26580100
7	2014-09-22	399.1000061	406.9159851	397.1300049	402.1520081	402.1520081	24127600

```
df = pd.read_csv('bitcoin_price.csv')
df.head()
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	2014-09-17	465.864014	468.174011	452.421997	457.334015	457.334015	21056800
1	2014-09-18	456.859985	456.859985	413.104004	424.440002	424.440002	34483200
2	2014-09-19	424.102997	427.834991	384.532013	394.795990	394.795990	37919700
3	2014-09-20	394.673004	423.295990	389.882996	408.903992	408.903992	36863600
4	2014-09-21	408.084991	412.425995	393.181000	398.821014	398.821014	26580100



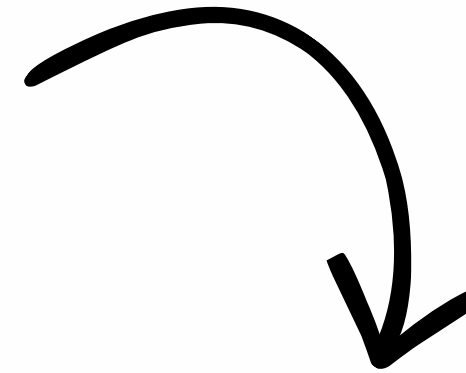
```
# Convert 'Date' To a Datetime and set as index
df['Date'] = pd.to_datetime(df['Date'], format = "%Y-%m-%d")
df.set_index('Date', inplace = True)
df.head()
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2014-09-17	465.864014	468.174011	452.421997	457.334015	457.334015	21056800
2014-09-18	456.859985	456.859985	413.104004	424.440002	424.440002	34483200
2014-09-19	424.102997	427.834991	384.532013	394.795990	394.795990	37919700
2014-09-20	394.673004	423.295990	389.882996	408.903992	408.903992	36863600
2014-09-21	408.084991	412.425995	393.181000	398.821014	398.821014	26580100



```
df1 = pd.read_csv("bitcoin_price.csv", index_col = "Date", parse_dates = True)
df1.index

DatetimeIndex(['2014-09-17', '2014-09-18', '2014-09-19', '2014-09-20',
                '2014-09-21', '2014-09-22', '2014-09-23', '2014-09-24',
                '2014-09-25', '2014-09-26',
                ...,
                '2023-12-21', '2023-12-22', '2023-12-23', '2023-12-24',
                '2023-12-25', '2023-12-26', '2023-12-27', '2023-12-28',
                '2023-12-29', '2023-12-30'],
              dtype='datetime64[ns]', name='Date', length=3392, freq=None)
```



```
df.resample('W').mean()
```

	Open	High	Low	Close	Adj Close	Volume
Date						
2014-09-21	429.916998	437.718195	406.624402	416.859003	416.859003	3.138068e+07
2014-09-28	410.650718	418.668993	399.377141	407.692566	407.692566	2.668180e+07
2014-10-05	369.774288	376.721000	353.207140	361.426570	361.426570	3.952256e+07

ANÁLISIS EXPLORATORIO

Promedio móvil: en cada fecha se promedian los últimos n valores, reduciendo el ruido día a día.

- **Suaviza la serie para que se vea más claro el nivel típico y la tendencia local.**
- **Ayuda a comparar y detectar cambios persistentes sin dejarse engañar por picos puntuales. No obstante, puede “retrasar” un poco la señal porque promedia varios días.**

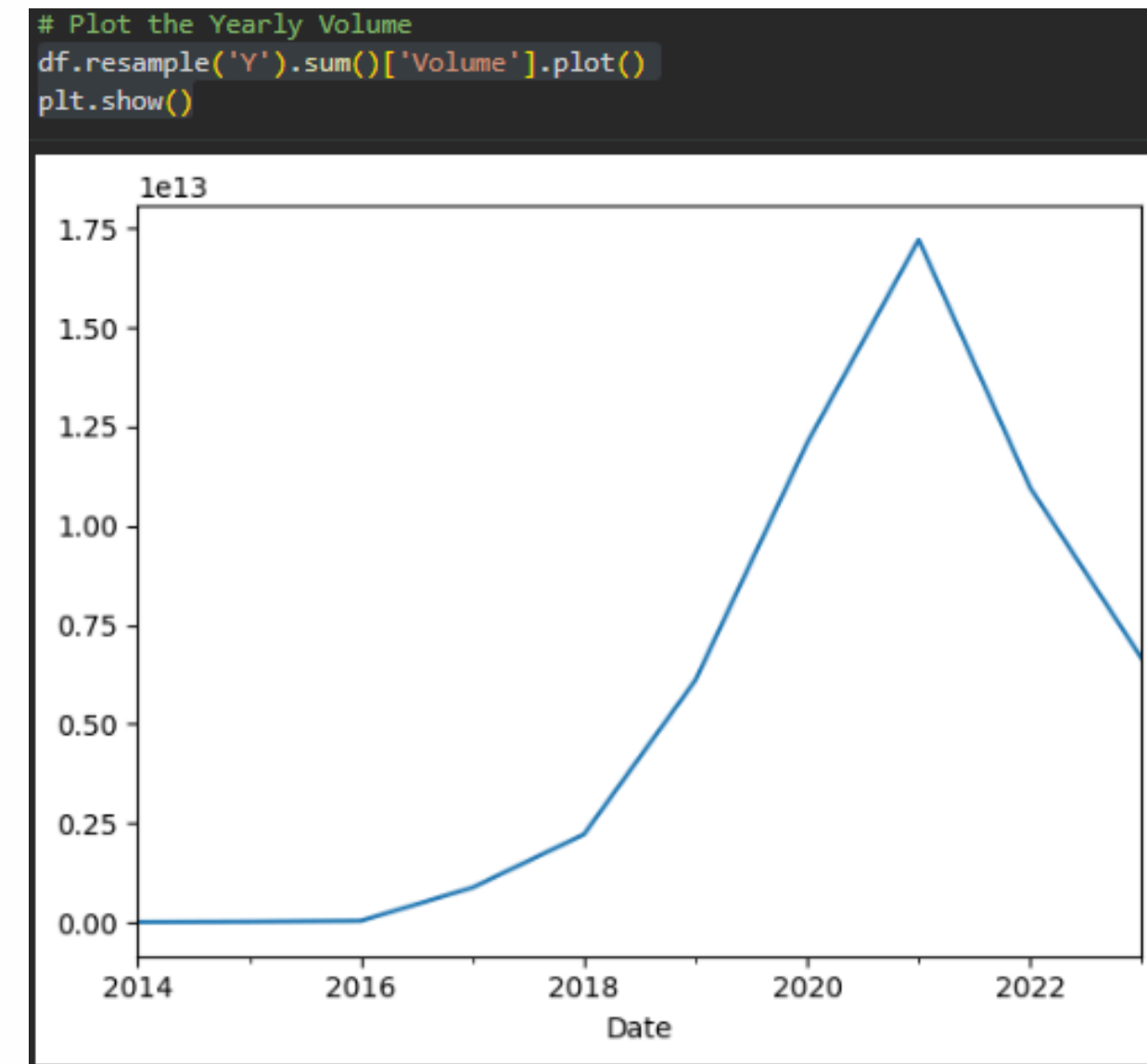
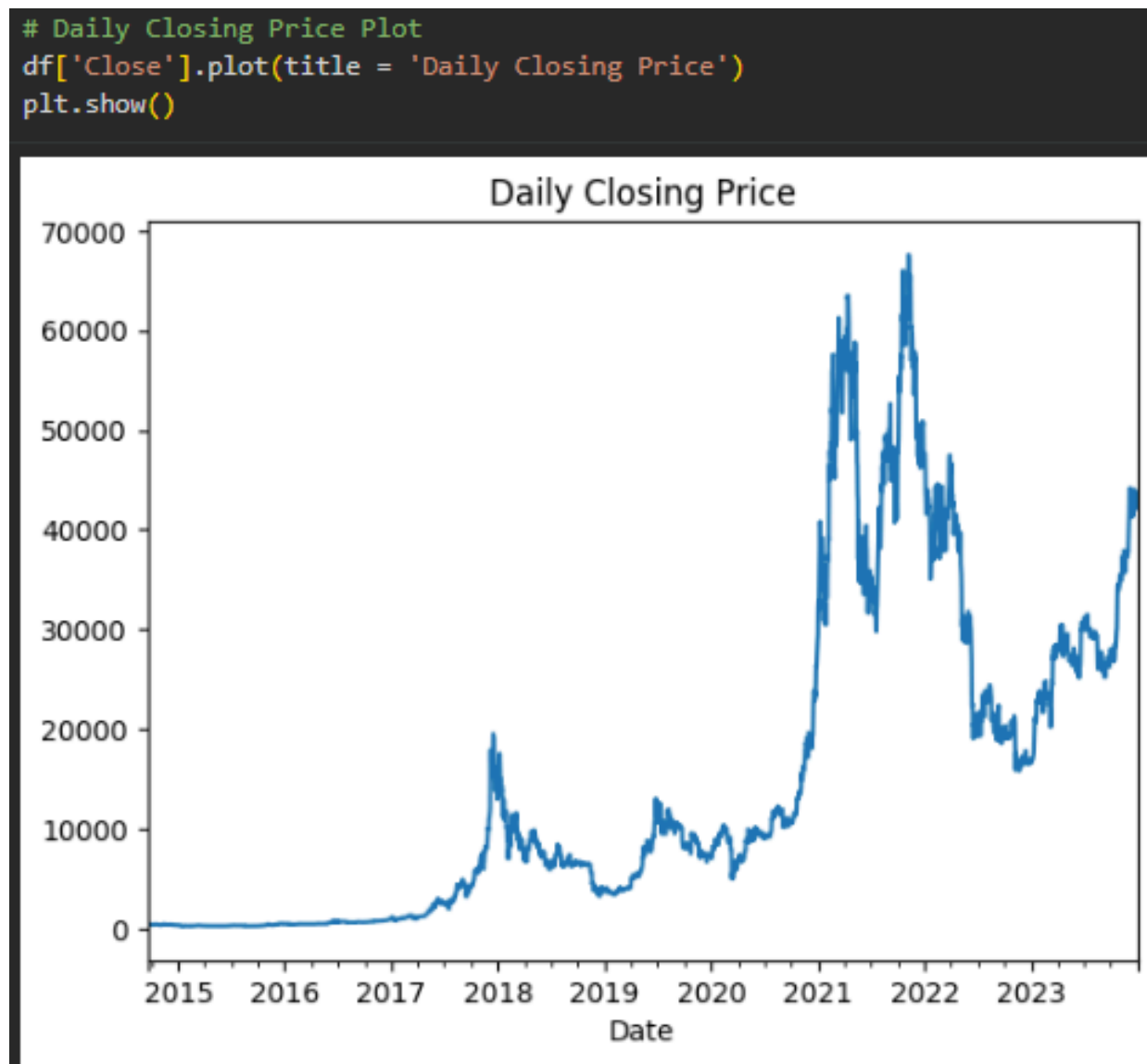
```
# Create a new column for 7- Day rolling average of the closing price
df['7_day_rolling']= df['Close'].rolling(window = 7).mean()
# Plot both the rolling average and closing values
df[['Close', '7_day_rolling']].loc['2023'].plot()
plt.show()
```

```
df.resample('M').mean()['Close'].idxmax()

Timestamp('2021-11-30 00:00:00', freq='M')
```



VISUALIZACIÓN



VISUALIZACIÓN

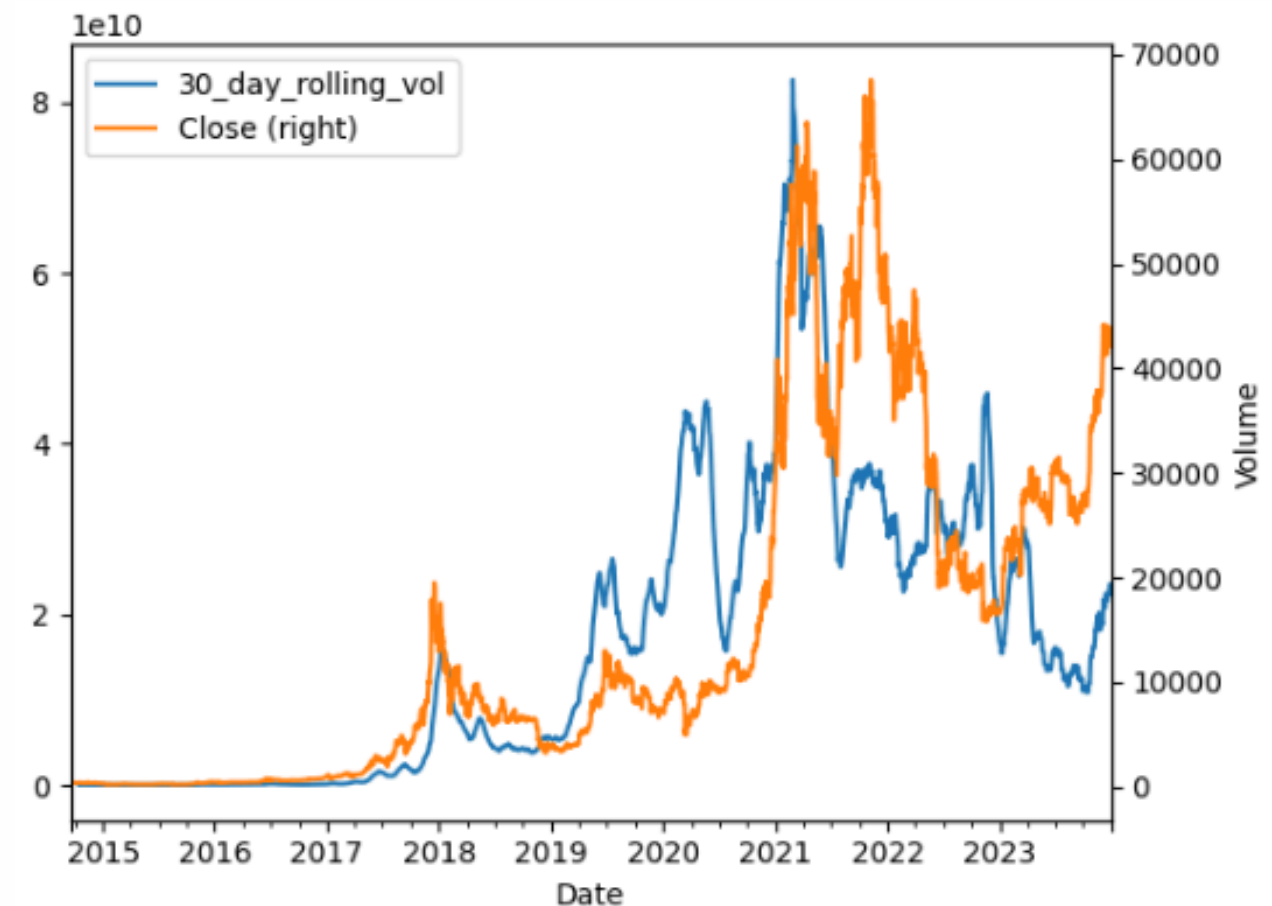
```
# Create a new column for 30- Day rolling average of the volume
df['30_day_rolling_vol'] = df['Volume'].rolling(window = 30).mean()

# Plotting Closing Price and 30-day rolling Volume
df['30_day_rolling_vol'].plot(legend = True)
ax = df['Close'].plot(secondary_y = True, legend = True)
ax.set_ylabel('Volume')
plt.show()
```

- Se comparan dos variables con escalas distintas, sin que una oculte a la otra.
- Analizar relaciones temporales, es decir, si cambios en una variable suelen coincidir o anticipar cambios en la otra.

```
df[['Close', '30_day_rolling_vol']].corr()
```

	Close	30_day_rolling_vol
Close	1.000000	0.750445
30_day_rolling_vol	0.750445	1.000000



PREPARACIÓN DE VARIABLES

Datos faltantes:

- **Eliminar (dropna).**
- **Valor fijo.**
- **Forward fill (ffill):** copia el último valor observado; asume que el proceso no cambia bruscamente.
- **Backward fill (bfill):** usa el siguiente valor disponible; útil cuando los faltantes están al inicio.
- **Media/mediana.**
- **Interpolación.**
- **Lag/estacional:** usa el valor del mismo punto del ciclo previo (día/semana).
- **Promedio por grupos temporales.**

PREPARACIÓN DE VARIABLES

```
# Identify missing values
df.isnull().sum()

# Fill Missing values
# method = 'bfill' - use next valid observation to fill gap.
df['30_day_rolling_vol'].fillna(method = 'bfill', inplace = True)

# Series.interpolate() - To estimate missing values based on the values around them.
# Interpolate missing 7 day rolling
df['7_day_rolling'].interpolate(inplace = True)
```

```
# Create 2 new columns for lagged values 1 and 2
df['closed_lag1'] = df['Close'].shift(1)
df['closed_lag2'] = df['Close'].shift(2)
```

```
# Extract Time Variables
df['year'] = df.index.year
df['month'] = df.index.month
df['day'] = df.index.day
df['weekday'] = df.index.day_name()
df['weekday_numeric'] = df.index.weekday

# 5 & 6 is for Saturday and Sunday
df['is_weekend'] = df.index.weekday > 4

df.head()
```



DESCOMPOSICIÓN ESTACIONARIA

- **Tendencia (Trend):** dirección general de la serie a largo plazo; cambia lentamente, no día a día. Puede ser creciente, decreciente o plana.
- **Estacionalidad (Seasonality):** patrones que se repiten periódicamente; constantes en el tiempo (diaria, semanal, anual).
- **Error (Residual):** parte no explicada por tendencia ni estacionalidad; comportamiento aleatorio sin patrón claro.

DESCOMPOSICIÓN ESTACIONARIA

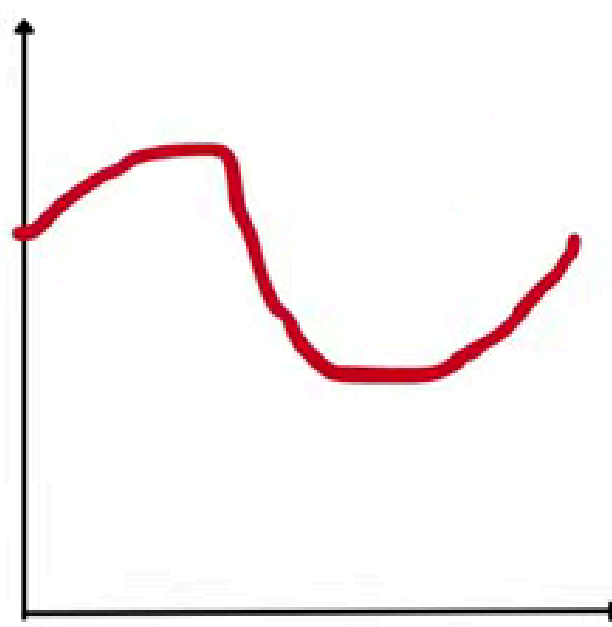
Trend

General direction
of the time series



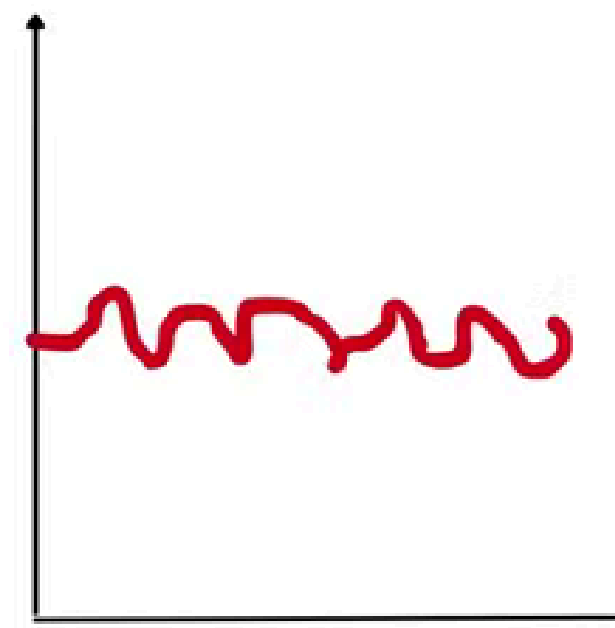
Seasonality

Seasonal cycles



Error

What is not
explained



ESTACIONALIDAD

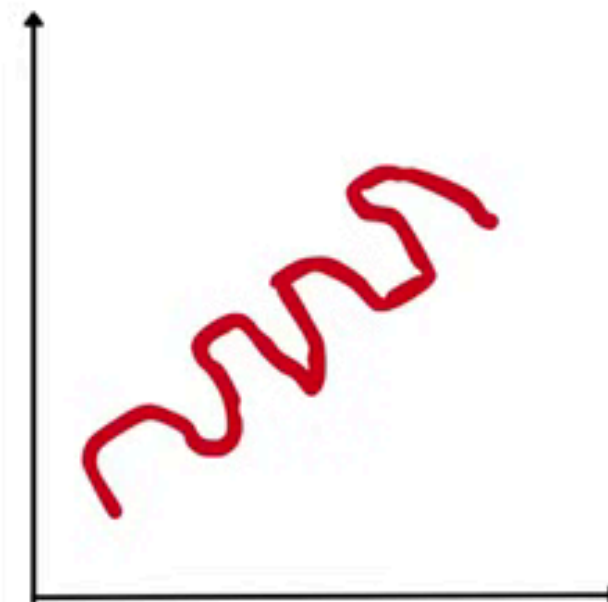
Tipos de estacionalidad:

- **Aditiva:** tiene amplitud constante, sin importar la tendencia.
- **Multiplicativa:** amplitud proporcional a la tendencia de la serie.

Ambas alrededor del nivel (valor de la tendencia en un punto t).

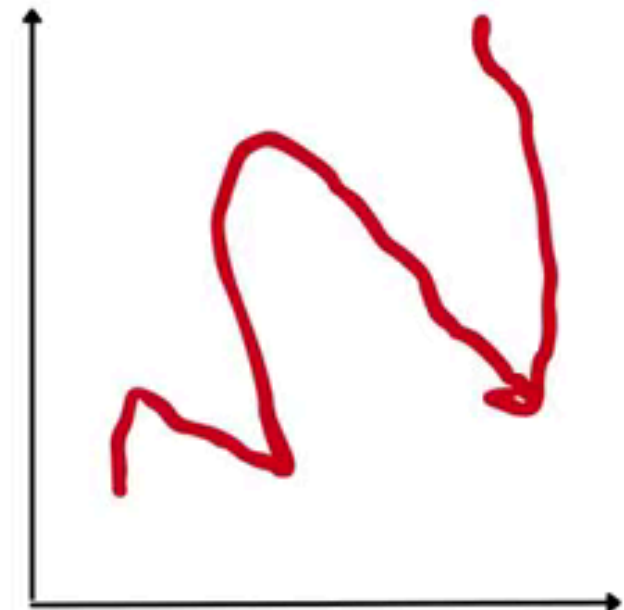
Additive

Seasonal cycles are constant in value



Multiplicative

Seasonal cycles are proportional to the trend



ESTACIONALIDAD

How to identify the seasonality type?

Unfortunately, there is not a statistical test to determine the seasonality type

Option 1: Data Visualization

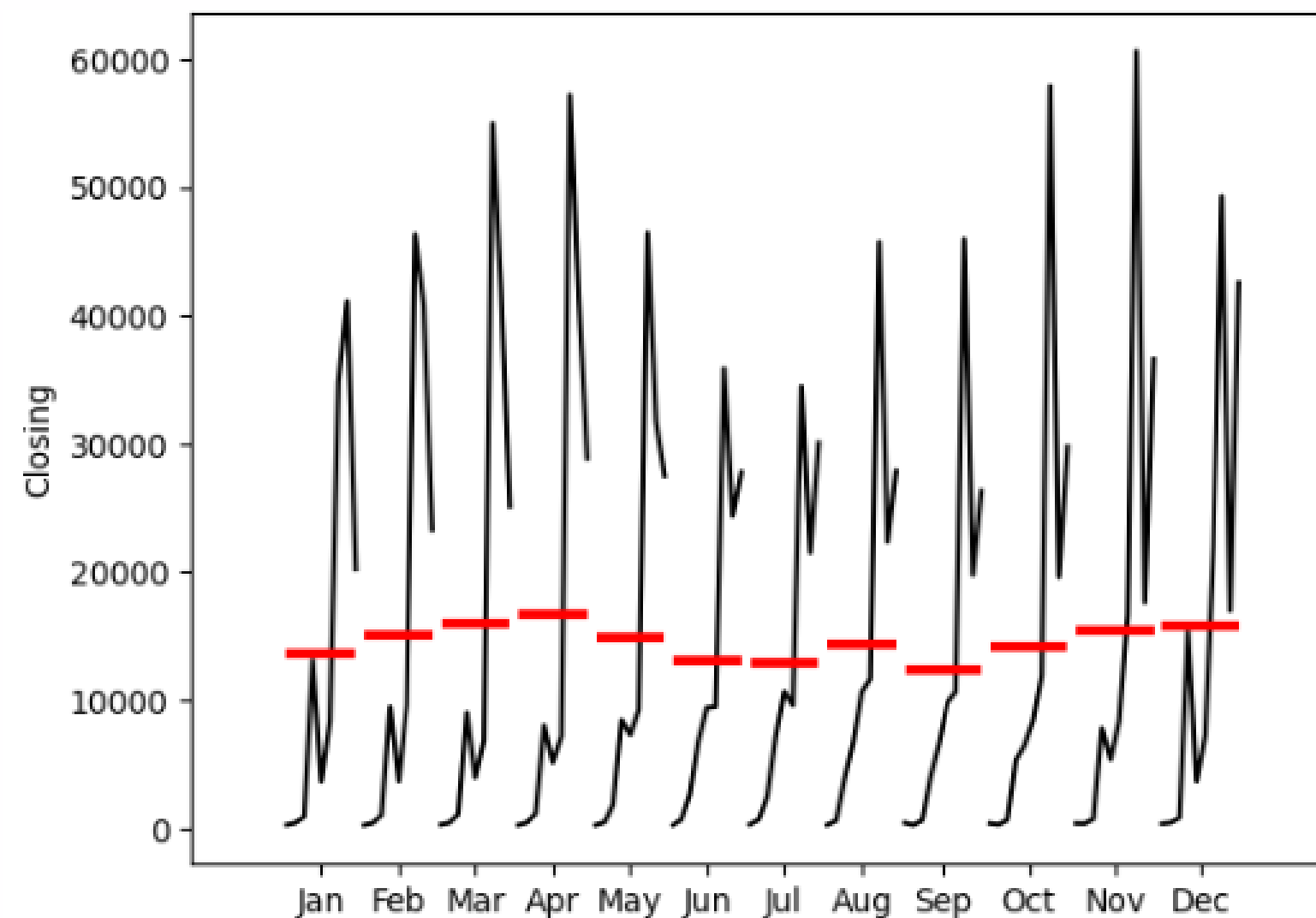
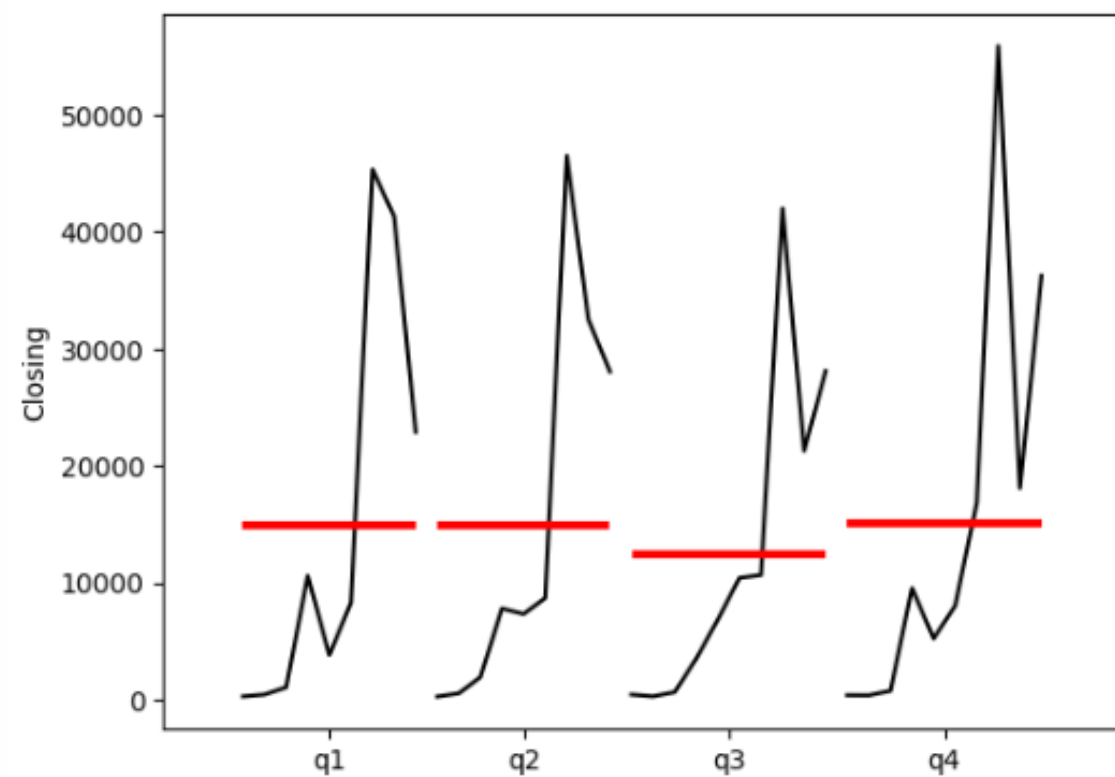
Option 2: Model Performance



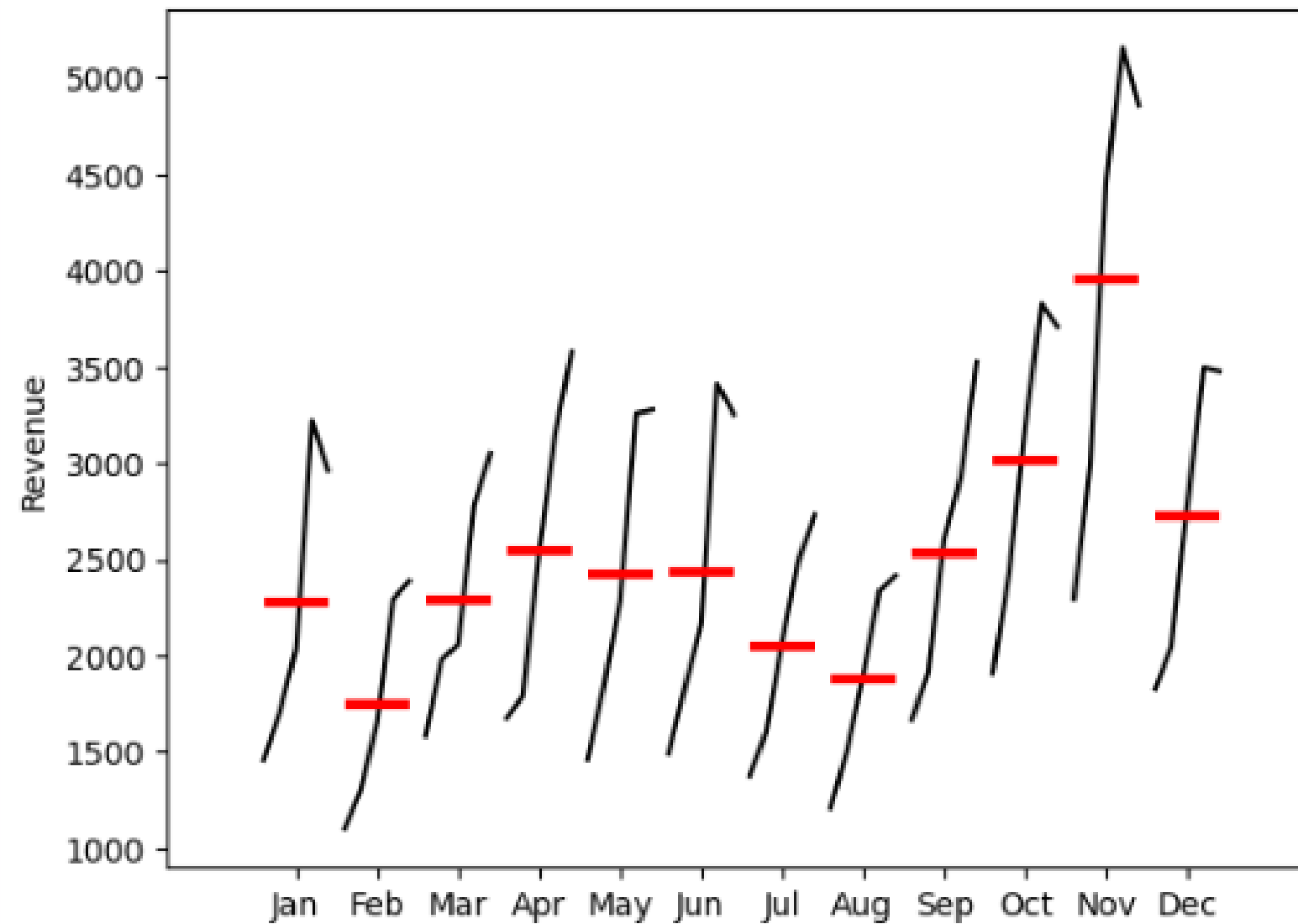
Implementar un modelo para cada tipo de estacionalidad y ver cuál se ajusta mejor a la serie de tiempo (recomendado).

ESTACIONALIDAD (VISUAL)

```
# Plotting the monthly seasonality of closing values  
month_plot(df['Close'].resample('M').mean(),  
           ylabel = 'Closing')  
plt.show()
```



ESTACIONALIDAD (VISUAL)



Con estos datos es más clara la estacionalidad.

Los test estadísticos los veremos después.

DESCOMPOSICIÓN ESTACIONARIA

```
# Seasonal Decomposition Plots for Bitcoin Data
# Multiplicative model type, and daily (365) period
decomposition = seasonal_decompose(df['Close'],
                                   model = 'mul',
                                   period = 365)

fig = decomposition.plot()
fig.set_size_inches(10,8)
plt.show()
```

Cada cuanto
se repite;
depende de
como estén
los datos.



- Aditivo: $y_t = T_t + S_t + R_t$
- Multiplicativo: $y_t = T_t \cdot S_t \cdot R_t$

$$R_t = y_t - T_t - S_t$$

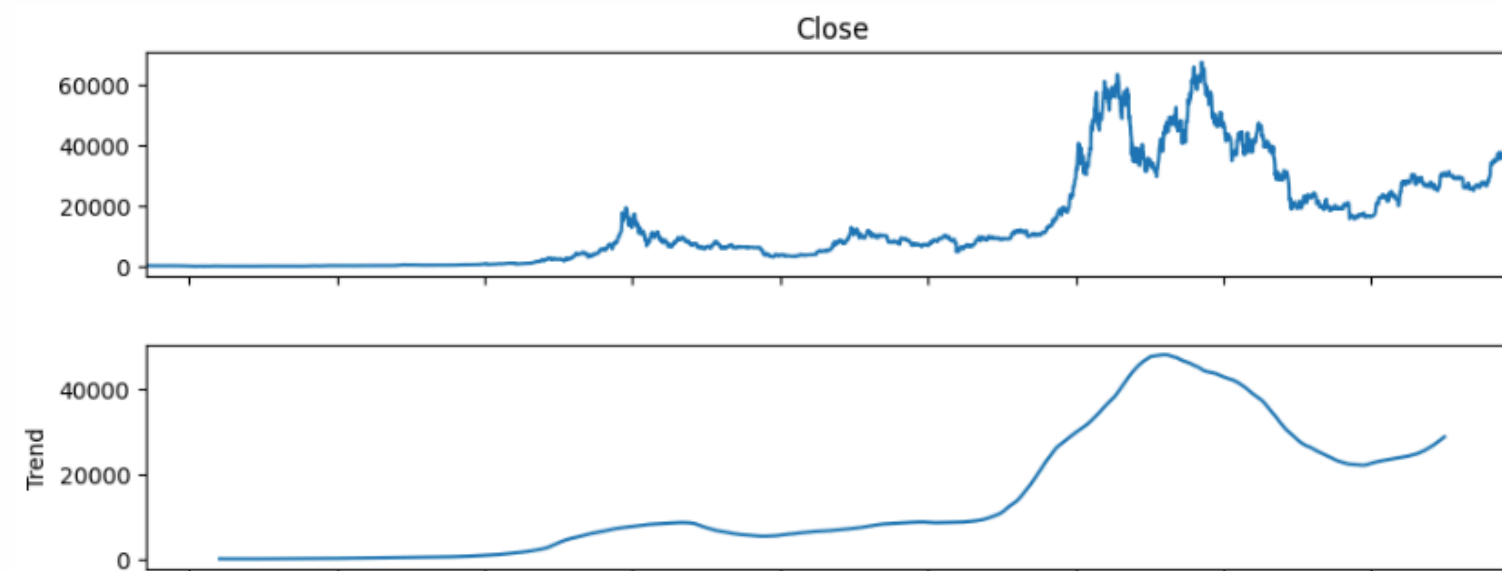
$$R_t = \frac{y_t}{T_t S_t}$$

La tendencia (T_t) es un promedio móvil; componente de baja frecuencia.

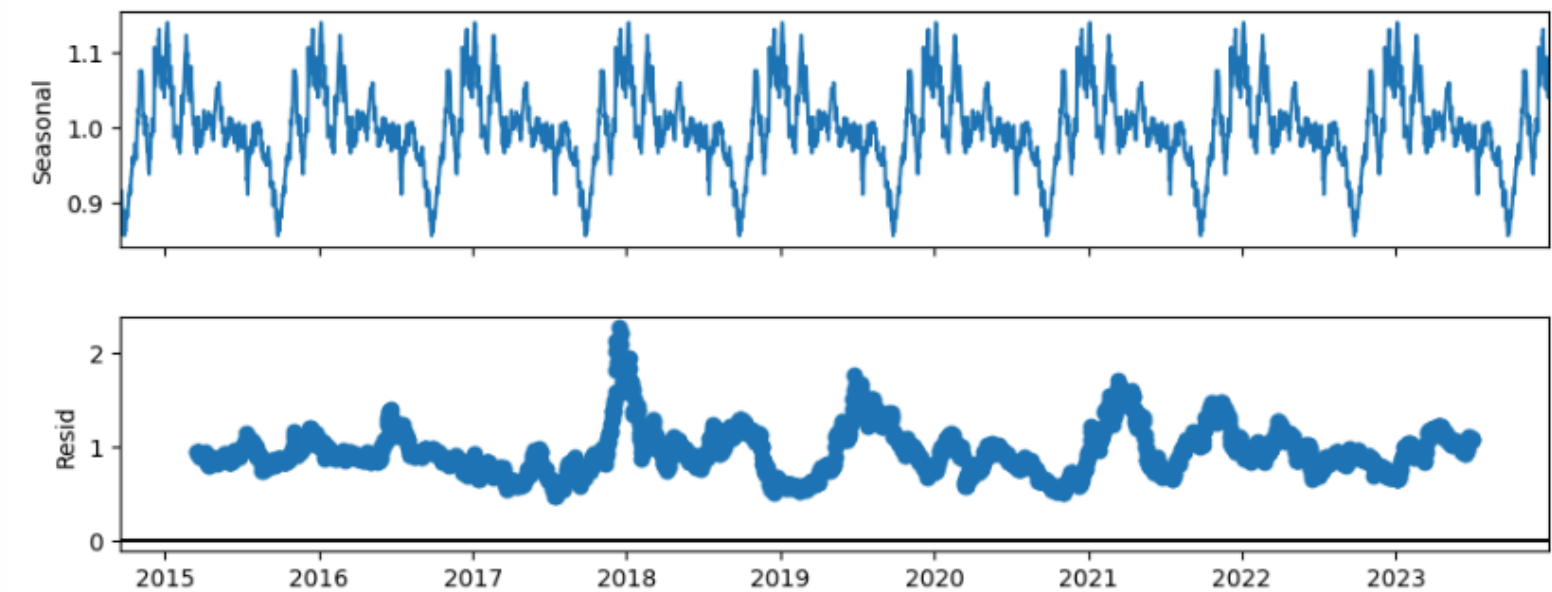
El comando saca cada componente, los va graficando y los va quitando de la serie hasta dejar el residuo.

El ruido ya no debería tener información/estructura de la serie.

DESCOMPOSICIÓN ESTACIONARIA



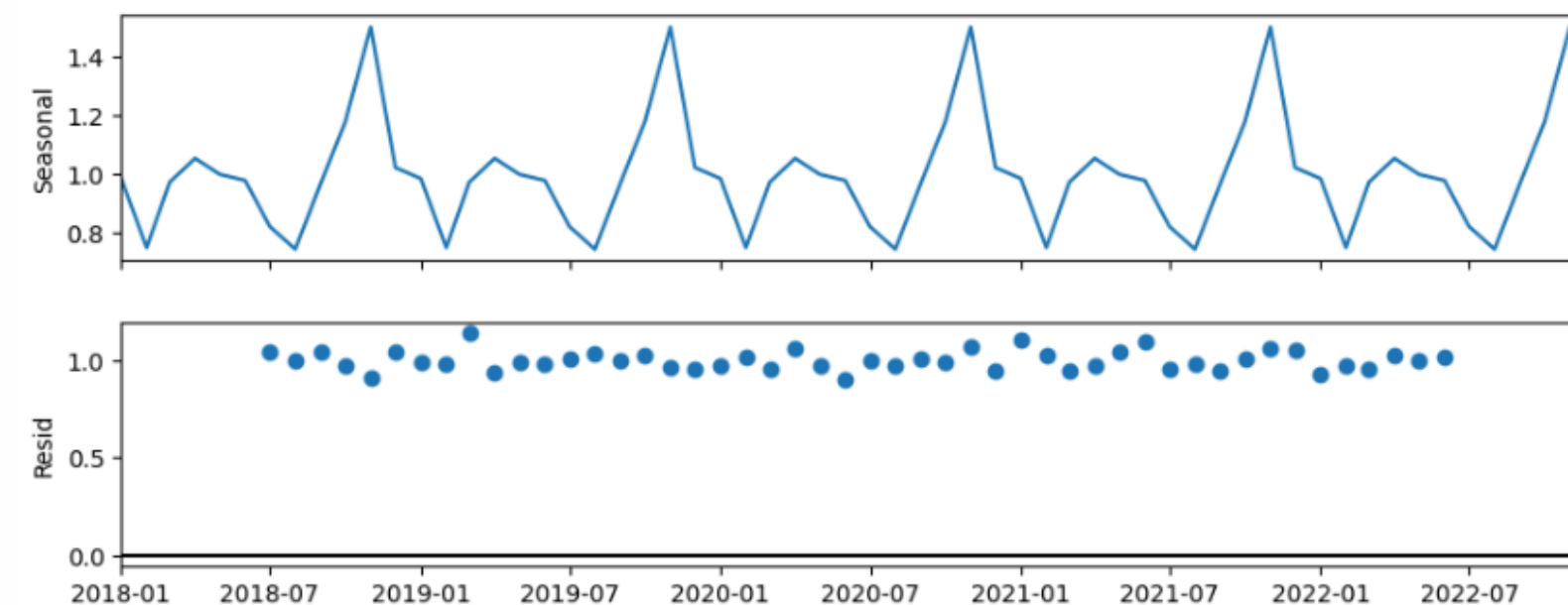
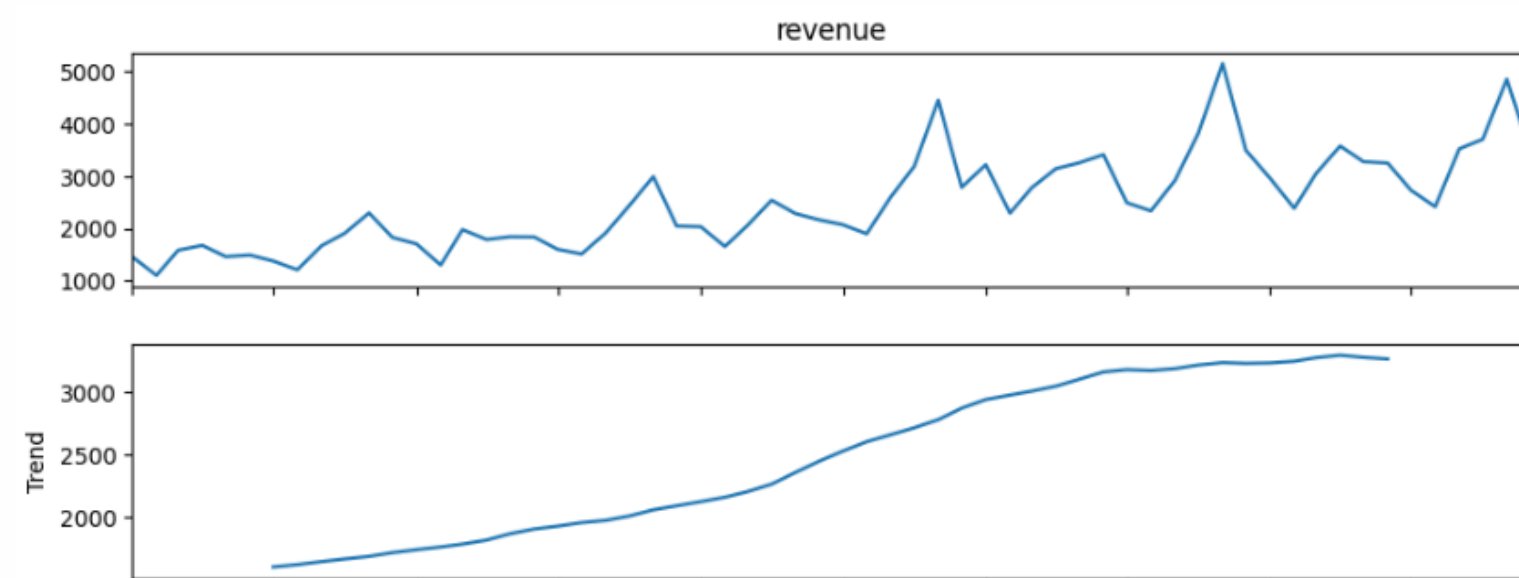
Con el tipo de estacionalidad mala, no la quitamos del todo y se filtra al residuo.



Con el periodo incorrecto, no va a detectar ninguna estacionalidad.

El residuo debería estar alrededor de 0 o 1 con los parámetros correctos.

DESCOMPOSICIÓN ESTACIONARIA



```
decomposition = seasonal_decompose(df_choco['revenue'],  
                                   model = 'mul',  
                                   period = 12)
```

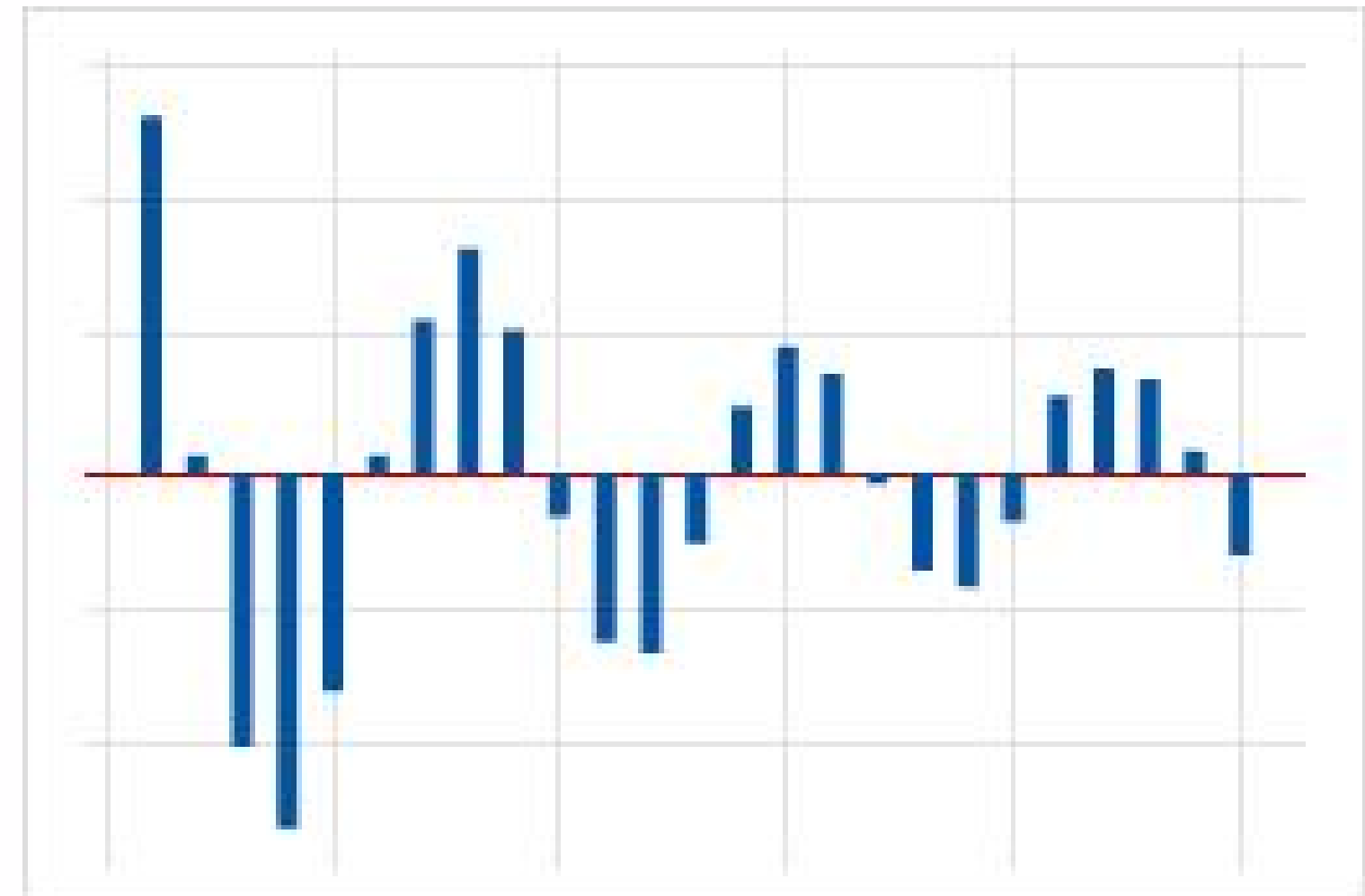
Con estos datos es más clara la descomposición y dichas características.

AUTOCORRELACIÓN

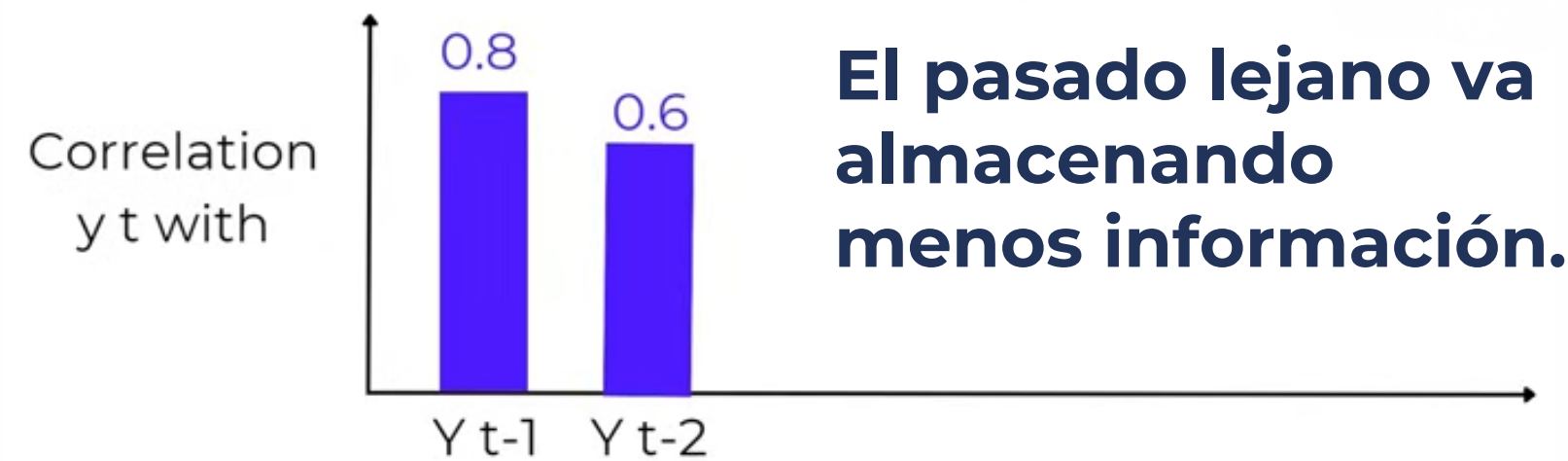
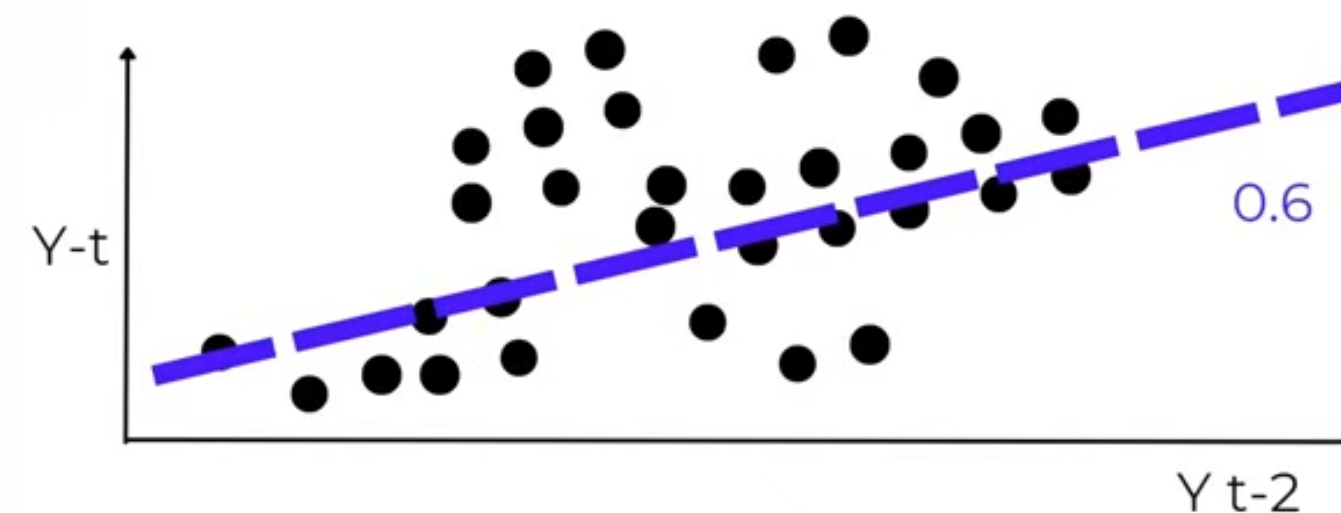
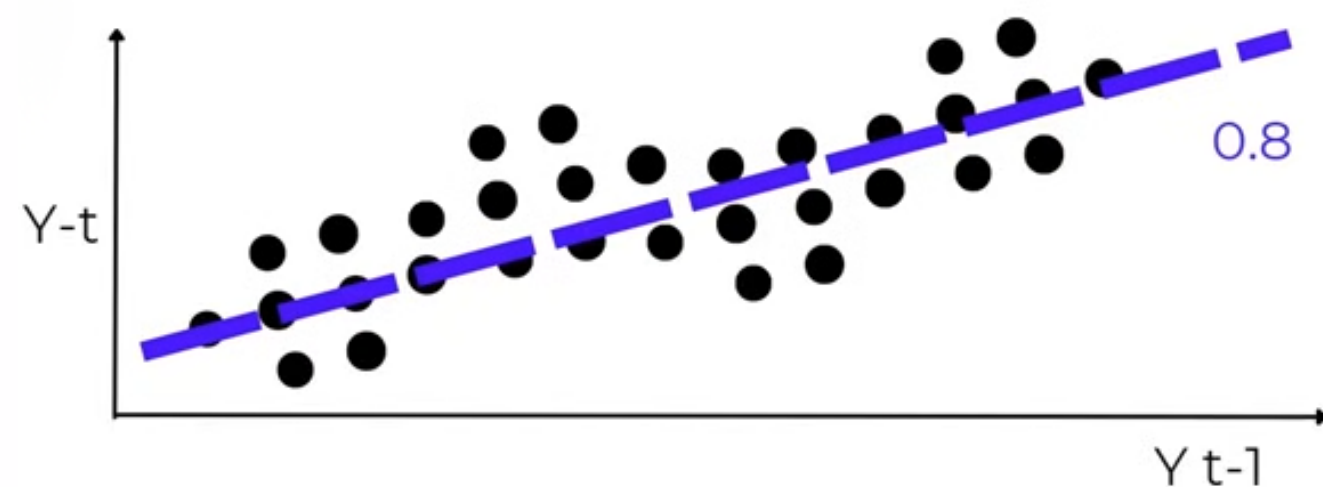
Es qué tanto depende el valor actual de la serie de sus valores pasados.

Nos ayuda a saber si hay información en el pasado que nos ayude a predecir el futuro.

El proceso consiste en correlacionar los valores de la serie de tiempo con sus mismos valores retardados.



AUTOCORRELACIÓN



El pasado lejano va almacenando menos información.

AUTOCORRELACIÓN

Interpretación:

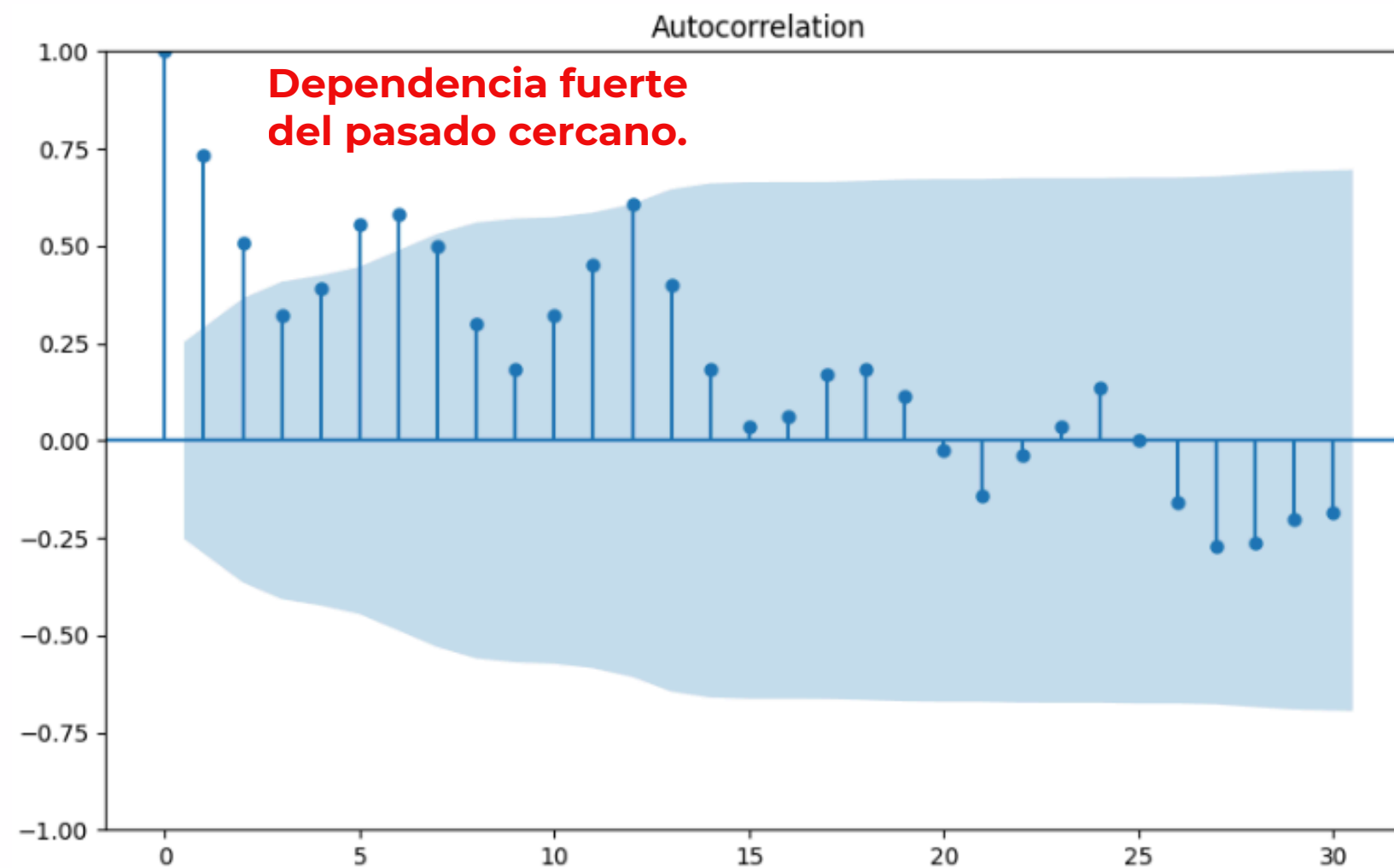
- Los valores van de -1 a 1 .
- 1 indica correlación positiva perfecta.
- -1 indica correlación negativa perfecta.
- 0 indica ausencia de correlación.

Gráfica ACF:

- Eje X: rezagos (lags).
- Eje Y: coeficiente de correlación.

```
fig, ax = plt.subplots(figsize = (10,6))  
plot_acf(df_choco['revenue'], lags = 30, ax = ax)  
plt.show()
```

AUTOCORRELACIÓN



- Los lags (rezagos) que sobrepasan la banda indican dependencia temporal estadísticamente significativa.

- Oscilaciones o picos → posible estacionalidad.

¿Cuántos lags son significativos?

¿Decae lento o rápido?

AUTOCORRELACIÓN PARCIAL

La información se propaga: la autocorrelación sola no distingue si la relación es directa, o heredada a través de otros lags (contiene relaciones indirectas).

La autocorrelación parcial mide si un rezago aporta información propia o si su efecto ya está explicado por rezagos más cercanos.

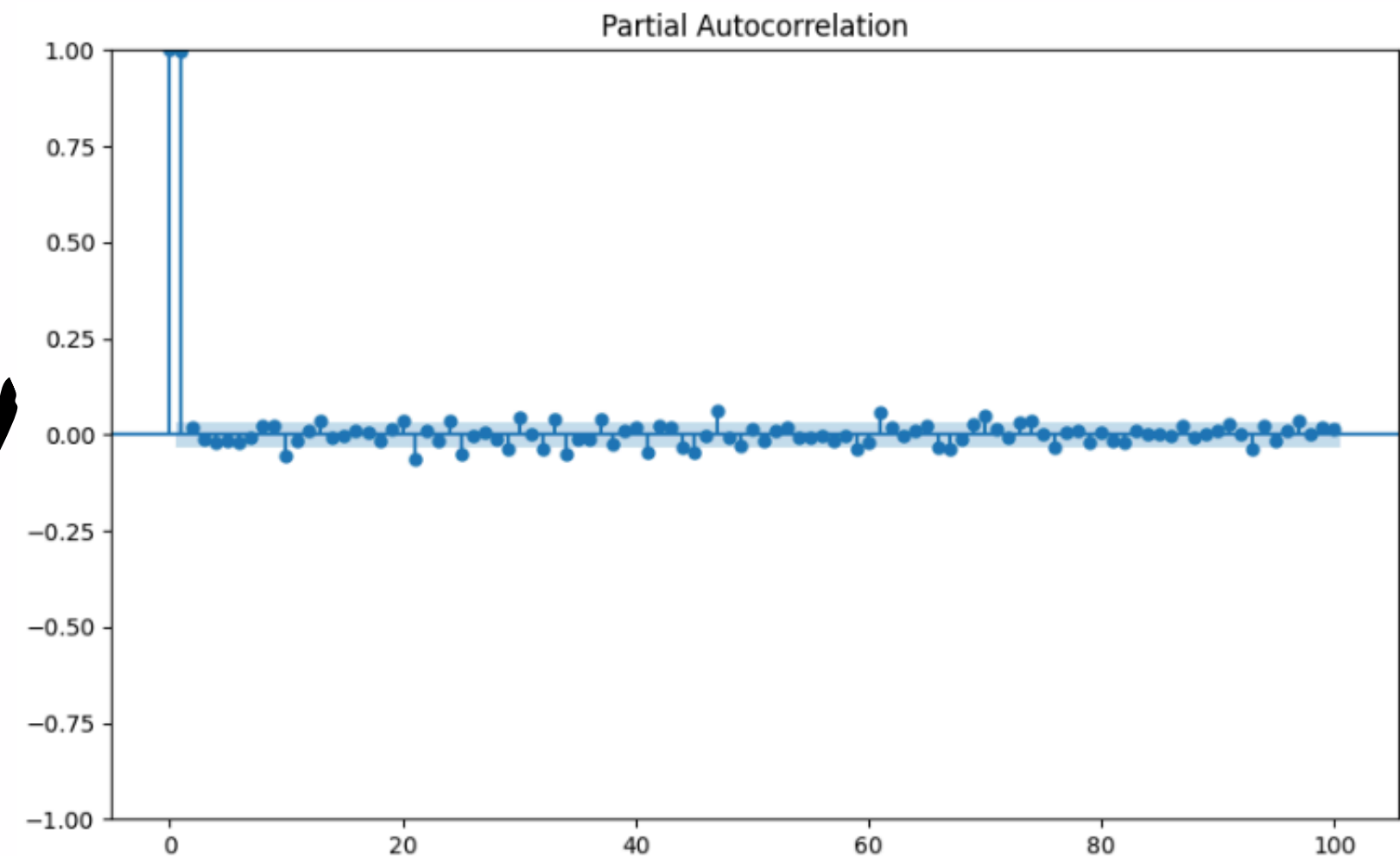


- **AC: detecta memoria total**
- **PAC: detecta memoria directa**

AUTOCORRELACIÓN PARCIAL

```
fig, ax = plt.subplots(figsize = (10,6))  
plot_pacf(df['Close'], lags = 100, ax = ax)  
plt.show()
```

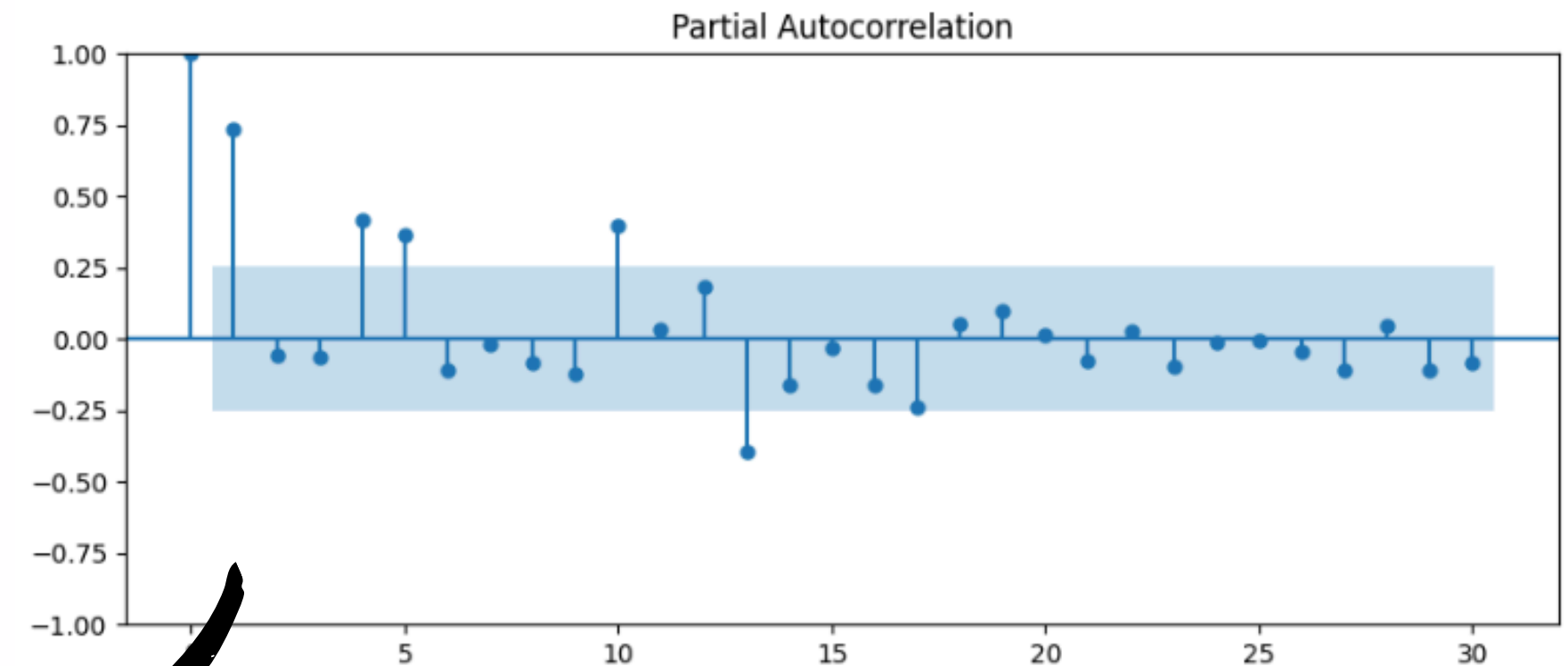
**Una caída rápida significa
que solo los valores recientes
influyen directamente en los
valores actuales.**



AUTOCORRELACIÓN PARCIAL

```
fig, ax = plt.subplots(figsize = (10,4))  
plot_pacf(df_choco['revenue'], lags = 30, ax = ax)  
plt.show()
```

Una caída lenta significa que los valores más antiguos siguen ejerciendo una influencia directa.





PRONÓSTICO

El principal desafío está en modelar e interpretar los errores (explicar e integrar los factores o regresores relevantes). Afectan la precisión del pronóstico.

- **Festivos**
- **Eventos masivos: conciertos o partidos.**
- **Lluvias fuertes (más uso del metro).**
- **Cambios de frecuencia**
- **Fallas.**
- **Hora del día (pico mañana / pico tarde).**
- **Día de la semana (laboral vs. fin de semana).**
- **Mes del año (estacionalidad anual).**





PRONÓSTICO

- **Pandemia → alteró gravemente las tendencias y los patrones estacionales establecidos.**

Eventos externos imprevistos pueden cambiar drásticamente la dinámica del mercado

El horizonte temporal importa: más años capturan patrones, pero datos muy antiguos pueden no representar la realidad actual y añadir ruido, por lo que a veces conviene excluirllos.

